

DELICIOUSTHINGS

Cahier des charges — MVP Request Marketplace

Intent-first · Agent-ready · 24 heures · 0 €

Version : 2.0

Objectif : validation produit en 24 heures

Stack cible : Laravel · Vue 3 · Inertia.js · TypeScript · SCSS · MySQL

1. Vision du MVP

Deliciousthings est une marketplace spécialisée dans les créations personnalisées réalisées par des artisans.

Le MVP ne cherche pas à construire un logiciel complet de gestion pour artisans ni une marketplace transactionnelle complète.

Il doit démontrer une capacité fondamentale :

Transformer une intention exprimée naturellement par un client en demande exploitable, la faire parvenir à des artisans pertinents et permettre au client de comparer leurs propositions.

Le produit est donc construit autour de la **Request**, et non autour d'un formulaire complexe ou d'un catalogue.

Le principe directeur est :

Describe → Confirm → Match → Propose → Choose

L'interface graphique est destinée aux humains dans le MVP, mais le modèle fonctionnel et les opérations métier doivent rester suffisamment structurés pour pouvoir être utilisés ultérieurement par des applications ou des agents IA externes.

2. Problème à résoudre

2.1 Côté client

Un client souhaite faire réaliser une création personnalisée.

Aujourd'hui, il doit généralement :

1. chercher des artisans ;
2. parcourir leurs profils ;
3. déterminer lesquels pourraient convenir ;
4. contacter plusieurs personnes ;
5. expliquer plusieurs fois son besoin ;
6. attendre les réponses ;
7. comparer des réponses parfois difficiles à comparer.

Le problème n'est donc pas simplement :

« Trouver un pâtissier. »

Le problème est :

« Faire comprendre rapidement mon besoin à plusieurs artisans capables de le réaliser et trouver la meilleure proposition. »

2.2 Côté artisan

L'artisan souhaite :

- trouver de nouveaux clients ;
- recevoir des demandes qu'il est réellement capable de satisfaire ;
- comprendre rapidement le besoin ;
- décider s'il souhaite répondre ;
- proposer un prix ;
- communiquer sa disponibilité ;
- éviter les demandes peu pertinentes.

Le problème n'est donc pas simplement :

« Obtenir plus de clients. »

Mais :

« Recevoir des opportunités pertinentes sans perdre du temps à qualifier des demandes inadaptées. »

3. Hypothèse produit principale

Si un utilisateur peut décrire son besoin avec ses propres mots et que Deliciousthings transforme cette description en une demande claire, vérifiable et exploitable par plusieurs artisans, alors :

la création d'une demande devient plus simple pour le client et plus pertinente pour l'artisan.

Le MVP doit tester cette hypothèse avant toute automatisation avancée.

4. Principe Product Rewired appliqué

Le MVP suit une logique volontairement stricte.

Étape 1 — Questionner

Ne pas considérer comme obligatoire :

- un formulaire détaillé ;
 - une recherche manuelle préalable ;
 - la sélection préalable d'un artisan ;
 - la connaissance précise du vocabulaire métier ;
 - la saisie de chaque attribut séparément.
-

Étape 2 — Supprimer

Sont supprimés du MVP :

- paiement ;
 - commande ;
 - devis complexe ;
 - négociation ;
 - messagerie temps réel ;
 - calendrier avancé ;
 - gestion CRM ;
 - communauté ;
 - avis ;
 - favoris ;
 - recommandations IA avancées ;
 - matching IA ;
 - administration complète ;
 - marketplace sophistiquée.
-

Étape 3 — Simplifier

Le parcours client devient :

Décrire

↓

Comprendre

↓

Confirmer

↓
Trouver
↓
Comparer

Le parcours artisan devient :

Recevoir
↓
Comprendre
↓
Proposer

Étape 4 — Accélérer

La priorité est de réduire :

- le nombre d'écrans ;
- le nombre de champs ;
- le nombre d'actions ;
- le temps nécessaire pour créer une demande ;
- le temps nécessaire pour comprendre une demande.

Étape 5 — Automatiser

L'automatisation IA n'est **pas une obligation du MVP**.

Elle pourra être ajoutée progressivement aux endroits où elle apporte une valeur démontrée :

Description
↓
Extraction
↓
Structuration
↓
Matching
↓
Résumé
↓
Comparaison

Le MVP doit donc être **AI-ready**, mais pas **AI-dependent**.

5. Le concept central : Request

La Request devient l'unité centrale de Deliciousthings.

Elle représente :

un besoin exprimé par un client et destiné à être satisfait par un ou plusieurs artisans.

Une Request peut être créée à partir d'un texte libre.

Exemple :

« Ich suche eine elegante Geburtstagstorte für 50 Personen am 28. September in Berlin.
Mein Budget liegt bei ungefähr 150 €. Sie sollte modern und nicht zu verspielt sein. »

Deliciousthings doit pouvoir représenter cette intention sous forme structurée :

Type	Torte
Occasion	Geburtstag
Date	28. September
Personnes	50
Lieu	Berlin
Budget	ca. 150 €
Style	Elegant / modern
Description	...

La représentation structurée est la donnée importante.

Le texte original doit également être conservé.

6. Parcours principal — Client

Étape 1 — Describe

L'utilisateur accède à une interface extrêmement simple.

Écran

Titre :

Was möchtest du anfertigen lassen?

Textarea principale :

Beschreibe, was du suchst.

Exemple :

„Ich suche eine elegante Geburtstagstorte für 50 Personen...“

L'utilisateur n'est pas obligé de connaître les catégories ou les champs nécessaires.

Action principale

Anfrage erstellen

7. Étape 2 — Understand

Après soumission, Deliciousthings présente ce qu'il a compris.

Exemple :

Das habe ich verstanden

Geburtstagstorte

- 50 Personen
- 1. September
- Berlin
- ca. 150 €
- elegant
- modern

Puis :

Fehlt etwas oder möchtest du etwas ändern?

Actions :

Bestätigen

Ändern

8. Règle importante : compréhension $\neq$ action

Le système ne doit jamais considérer une interprétation comme une instruction définitive.

Architecture logique :

```
User intent
  ↓
Interpretation
  ↓
Confirmation
  ↓
Action
```

Cette distinction est essentielle pour une future utilisation par des agents IA.

Un agent pourra également :

```
understand_request()
  ↓
confirm_with_user()
  ↓
submit_request()
```

au lieu de déclencher directement une action irréversible.

9. Étape 3 — Match

Une fois la demande confirmée, Deliciousthings recherche les artisans pertinents.

Pour le MVP, le matching reste déterministe.

Critères possibles :

1. catégorie ;
2. localisation ;
3. capacité ;
4. disponibilité déclarée ;

5. compatibilité avec le type de demande.

Le matching avancé par IA est hors périmètre.

10. Étape 4 — Propose

Deliciousthings présente les artisans correspondant à la demande.

L'utilisateur ne doit pas avoir à rechercher individuellement chaque artisan.

Exemple :

3 passende Anbieter gefunden

Chaque résultat affiche uniquement les informations nécessaires :

- nom ;
- localisation ;
- spécialité ;
- courte description ;
- quelques créations ;
- indication de disponibilité si disponible.

CTA :

Anfrage senden

11. Envoi d'une Request

L'utilisateur peut envoyer la même Request à plusieurs artisans.

La Request originale reste identique.

Cela permet à Deliciousthings de devenir le point de structuration commun entre :

Client
↓
Request
↓
Artisan A
Artisan B
Artisan C

Chaque artisan reçoit sa propre relation avec la Request.

12. Étape 5 — Propositions

L'artisan peut répondre à la Request.

La proposition MVP contient :

- prix proposé ;
- message ;
- disponibilité ;
- éventuellement une date estimée.

Exemple :

```
Preis
145 €

Verfügbarkeit
Ja

Nachricht
Ich kann die Torte am 28. September vorbereiten...
```

CTA :

Angebot senden

13. Comparaison côté client

Le client consulte les réponses reçues.

Les propositions doivent être présentées de manière structurée afin de faciliter la comparaison.

Exemple :

Artisan	Prix	Disponibilité	Réponse
Artisan A	145 €	Disponible	Voir
Artisan B	160 €	Disponible	Voir
Artisan C	130 €	À confirmer	Voir

L'objectif n'est pas encore de construire un moteur de recommandation.

L'objectif est :

rendre les propositions comparables.

14. Choix

Le MVP permet au client de sélectionner une proposition.

Cette action ne déclenche pas de paiement.

Elle représente uniquement :

la proposition préférée du client.

La transaction réelle reste hors périmètre.

15. Parcours artisan

L'artisan dispose d'un espace minimal.

Dashboard

Il voit :

- nouvelles Requests ;
 - Requests en cours ;
 - Requests auxquelles il a répondu.
-

Request detail

L'artisan peut consulter :

- description originale ;
- informations structurées ;
- date ;
- quantité ;
- localisation ;
- budget indicatif ;
- informations complémentaires.

Actions :

Anfrage beantworten

Ablehnen

16. Profil artisan minimal

Un artisan peut disposer de :

- nom ;
- description ;
- ville ;
- spécialités ;
- créations ;
- disponibilité générale ;
- informations de contact nécessaires.

Le profil doit être suffisamment riche pour inspirer confiance, mais ne doit pas devenir un projet de portfolio.

17. Modèle de données MVP

User

```
id
first_name
last_name
email
password
role
timestamps
```

Artisan

```
id
user_id
name
slug
bio
city
country
is_published
timestamps
```

Creation

```
id
artisan_id
title
description
image
timestamps
```

Request

```
id
user_id
description

type
occasion
event_date
guest_count
location
budget_min
budget_max
style

status

timestamps
```

Le champ `description` conserve toujours l'intention originale de l'utilisateur.

Les autres champs représentent l'interprétation structurée.

RequestParticipant

Relation entre une Request et un artisan.

```
id
request_id
artisan_id
status
sent_at
responded_at
timestamps
```

Statuts :

```
pending
sent
viewed
declined
responded
selected
```

Proposal

```
id
request_id
artisan_id
price
availability
message
status
timestamps
```

Statuts :

```
draft
submitted
withdrawn
selected
rejected
```

18. États principaux

Les états doivent être explicites.

Request

```
draft
confirmed
sent
responses_pending
completed
cancelled
```

RequestParticipant

```
pending
sent
viewed
declined
responded
selected
```

Proposal

```
draft
submitted
withdrawn
selected
rejected
```

L'utilisation d'états explicites facilite à la fois :

- l'interface ;
- les règles métier ;
- les tests ;
- une future API ;
- l'utilisation par des agents.

19. Architecture applicative

La stack reste :

```
Laravel
Vue 3
Inertia.js
TypeScript
SCSS
MySQL
```

Architecture :

```
Vue / Inertia
  ↓
Controllers
  ↓
Actions / Application Services
  ↓
Domain rules
```

↓
Eloquent
↓
MySQL

La logique métier importante ne doit pas être enfermée dans les composants Vue.

20. Actions métier recommandées

Les opérations importantes doivent être isolées.

CreateRequest
ConfirmRequest
FindMatchingArtisans
SendRequest
DeclineRequest
CreateProposal
SubmitProposal
SelectProposal

Cela permet d'avoir demain :

Vue
↓
CreateRequest

mais également :

API
↓
CreateRequest

et éventuellement :

AI Agent
↓
CreateRequest

sans réécrire les règles métier.

21. Agent-readiness

Le MVP ne comporte aucune interface IA obligatoire.

Cependant, chaque opération importante doit pouvoir être décrite comme une capacité indépendante.

Exemple :

```
create_request
```

Entrée :

```
{  
  "description": "..."  
}
```

Résultat conceptuel :

```
{  
  "id": 123,  
  "status": "draft"  
}
```

Puis :

```
confirm_request
```

```
find_matching_artisans
```

```
send_request
```

```
get_proposals
```

```
select_proposal
```

Cette structure prépare naturellement une future API ou interface MCP.

Aucun MCP server ne doit être développé pour le MVP.

22. Pourquoi cette architecture est agent-ready

Un agent externe pourrait à terme effectuer :

```
Utilisateur
  ↓
Agent IA
  ↓
create_request()
  ↓
confirm_request()
  ↓
find_matching_artisans()
  ↓
send_request()
  ↓
get_proposals()
  ↓
compare
  ↓
Utilisateur
```

Deliciousthings n'a donc pas besoin de devenir lui-même l'agent.

Il fournit une capacité spécialisée :

trouver et connecter une demande de création personnalisée à des artisans capables d'y répondre.

23. Interface utilisateur

L'interface doit être :

- calme ;
- chaleureuse ;
- claire ;
- peu chargée ;
- mobile-first ;
- fortement hiérarchisée ;
- orientée vers une action principale ;
- compréhensible sans apprentissage.

La priorité n'est pas de montrer toutes les possibilités.

La priorité est de rendre évident :

« Dis-moi ce dont tu as besoin. »

24. Référence visuelle unique

Linear

Référence unique pour l'interface applicative.

Utiliser Linear comme référence pour :

- sobriété ;
- hiérarchie ;
- densité ;
- navigation ;
- états ;
- listes ;
- cartes ;
- feedback ;
- cohérence des composants ;
- gestion des espaces.

Ne pas reproduire son identité visuelle.

La personnalité de Deliciousthings doit rester propre :

- tons chauds ;
- beige / blanc cassé ;
- brun-noir ;
- rose/prune ;
- Fraunces pour les titres ;
- Inter pour l'interface.

Le document précédent définissait déjà cette direction typographique et chromatique : Fraunces pour les titres, Inter pour l'interface, fond chaud, texte brun-noir et accent rose/prune.

Aucune autre référence visuelle n'est nécessaire pour le MVP.

25. Écrans minimum

P0 — Client

1. Landing / Entry

Objectif :

commencer une Request.

Élément principal :

Was möchtest du anfertigen lassen?
[textarea]
[Anfrage erstellen]

2. Request Understanding

Affiche :

- description originale ;
- données comprises ;
- possibilité de correction ;
- confirmation.

3. Matching Results

Affiche :

- nombre d'artisans pertinents ;
- cartes artisan ;
- informations essentielles ;
- sélection.

4. Request Sent

Affiche :

- artisans contactés ;
- statut ;
- attente des réponses.

5. Proposals

Affiche :

- propositions ;
- prix ;
- disponibilité ;
- messages ;
- comparaison.

6. Proposal Detail

Affiche le détail d'une proposition.

CTA :

Auswählen

P0 — Artisan

7. Artisan Dashboard

Affiche :

- nouvelles demandes ;
 - demandes en cours ;
 - réponses envoyées.
-

8. Request Detail

Affiche :

- besoin original ;
 - données structurées ;
 - contexte ;
 - actions.
-

9. Proposal Create

Formulaire minimal :

- prix ;
 - disponibilité ;
 - message.
-

P1

- profil public artisan ;
- créations ;
- authentification complète ;
- édition du profil.

Ces éléments sont nécessaires seulement dans la mesure où ils permettent de rendre le parcours vertical démontrable.

26. Composants UI MVP

Réutiliser ou adapter les composants existants lorsque leur qualité est suffisante.

Composants prioritaires :

```
AppButton
AppInput
AppTextarea
AppCard
StatusBadge
PageContainer
PageHeader

RequestCard
ArtisanCard
ProposalCard

EmptyState
LoadingState
```

Les composants secondaires peuvent rester simples :

- Modal ;
- Dropdown ;
- Toast ;
- Avatar ;
- Tooltip.

Le cahier des charges précédent appliquait déjà cette logique de priorité : les composants fondamentaux étaient à construire, tandis que Modal, Dropdown, Avatar, Toast et ServiceBadge pouvaient rester secondaires.

27. Ce qui est volontairement hors périmètre

Ne pas implémenter :

IA

- chatbot ;
- agent autonome ;
- LLM obligatoire ;
- embeddings ;
- vector database ;
- RAG ;
- AI matching ;
- AI recommendations ;

- MCP ;
- orchestration multi-agent.

Marketplace

- paiement ;
- commission ;
- checkout ;
- réservation ;
- contrat ;
- facturation ;
- négociation ;
- système d'enchères.

Communication

- messagerie temps réel ;
- notifications push ;
- chat avancé.

Artisan

- CRM ;
- calendrier complet ;
- gestion des tâches ;
- comptabilité ;
- gestion de production.

Social

- communauté ;
- commentaires ;
- followers ;
- likes ;
- favoris ;
- reviews.

Administration

- back-office complet ;
- modération avancée ;
- analytics avancés.

28. Seed data

Le MVP doit être démontrable immédiatement.

Créer des artisans fictifs représentatifs :

5–10 artisans

avec :

- différentes villes ;
- différentes spécialités ;
- différentes capacités ;
- plusieurs créations ;
- différentes disponibilités.

Créer également quelques Requests et Proposals permettant de tester les états principaux.

29. Validation serveur

Toutes les données critiques doivent être validées côté Laravel.

Exemples :

```
description : required
event_date : nullable/date
guest_count : nullable/integer
budget_min : nullable/numeric
budget_max : nullable/numeric
location : nullable/string
```

Les règles métier doivent également être protégées :

- une Request non confirmée ne peut pas être envoyée ;
 - une Request annulée ne peut plus recevoir de proposition ;
 - une proposition ne peut être soumise qu'une seule fois ;
 - seul le propriétaire peut sélectionner une proposition ;
 - seul l'artisan destinataire peut répondre à une Request.
-

30. Responsive

Approche mobile-first.

Le MVP doit fonctionner correctement :

- mobile ;
- tablette ;
- desktop.

Aucune variante responsive complexe.

Priorité :

mobile
↓
desktop

Les écrans principaux doivent rester utilisables avec une seule main sur mobile lorsque cela est pertinent.

31. Critères de réussite du MVP

Le MVP est considéré comme réussi techniquement si un utilisateur peut :

1. arriver sur Deliciousthings
2. décrire son besoin en langage naturel
3. voir ce que Deliciousthings a compris
4. confirmer la Request
5. voir des artisans correspondants
6. envoyer la Request
7. recevoir plusieurs propositions
8. comparer les propositions
9. sélectionner une proposition

Et si un artisan peut :

1. se connecter
2. voir les Requests pertinentes
3. ouvrir une Request
4. comprendre le besoin
5. proposer un prix
6. indiquer sa disponibilité
7. envoyer sa proposition

32. Critère produit fondamental

La question à poser pendant toute l'implémentation est :

« Est-ce que cette fonctionnalité améliore directement la transformation d'une intention en proposition pertinente ? »

Si la réponse est non :

TODO après MVP.

Cette règle reprend le principe du prototype précédent selon lequel une amélioration qui ne débloque pas le parcours principal doit être repoussée.

33. Ordre d'implémentation — 24 heures

Phase 1 — Foundation

1. Vérifier architecture existante
2. Vérifier authentification
3. Vérifier User
4. Préparer Request
5. Préparer RequestParticipant
6. Préparer Proposal

Phase 2 — Core Request

7. Request migration
8. Request model
9. CreateRequest
10. RequestController
11. Create Request page
12. Request detail
13. Validation

Phase 3 — Understanding

14. Structured Request representation
15. Confirmation UI
16. Correction
17. Request state management

Pour le MVP sans coût :

la compréhension structurée peut être initialement déterministe ou saisie/complétée par des champs minimaux après la description.

L'objectif est de valider le **workflow**, pas la qualité d'un LLM.

Phase 4 — Matching

- 18. Artisan model
- 19. Artisan seeders
- 20. Matching query
- 21. Matching results
- 22. ArtisanCard
- 23. Send Request

Phase 5 — Proposals

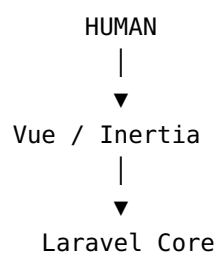
- 24. Proposal migration
- 25. Proposal model
- 26. Artisan Request detail
- 27. Create Proposal
- 28. Client proposal list
- 29. Proposal comparison
- 30. Select Proposal

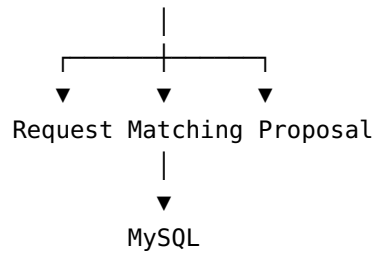
Phase 6 — Polish

- 31. Empty states
- 32. Loading states
- 33. Error states
- 34. Validation messages
- 35. Responsive polish
- 36. Status styling
- 37. Final vertical walkthrough

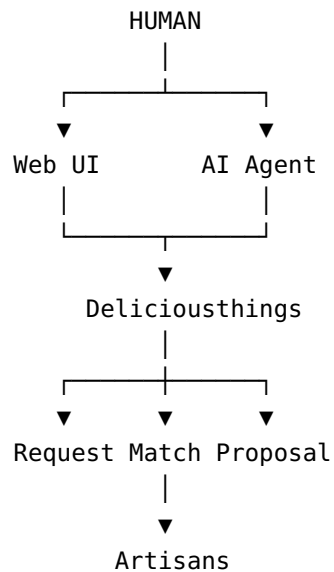
34. Architecture future

Le MVP :





Évolution :



Le principe architectural est donc :

L'interface peut changer. Le cœur métier ne doit pas dépendre de l'interface.

35. Évolution potentielle après validation

V1 — Natural Language

```

User
↓
Description
↓
AI extraction
↓
Structured Request
  
```

V2 — Agent-assisted marketplace

```
User
↓
AI Agent
↓
Deliciousthings
↓
Search
↓
Requests
↓
Proposals
```

V3 — Agentic transaction

```
User
↓
Agent
↓
Understand
↓
Search
↓
Contact
↓
Compare
↓
Recommend
↓
Human approval
↓
Transaction
```

36. Définition finale du MVP

Le MVP Deliciousthings n'est pas :

« un logiciel de gestion pour pâtisseries avec de l'IA ».

Il n'est pas non plus :

« un catalogue de pâtisseries ».

Il est :

un Request Marketplace permettant à un utilisateur d'exprimer simplement un besoin de création personnalisée, de le structurer, de le transmettre à plusieurs artisans pertinents et de comparer leurs propositions.

La première interface est humaine.

Le modèle métier est structuré.

Les opérations sont explicites.

L'IA est différée.

L'architecture reste prête à accueillir des agents.

37. Principe directeur final

Ne pas construire l'IA.

Ne pas construire le marketplace complet.

Ne pas construire le SaaS artisan.

Construire la capacité fondamentale :

« J'ai besoin de quelque chose de personnalisé → Deliciousthings trouve des personnes capables de le réaliser et me permet de comparer leurs réponses. »

Tout ce qui ne sert pas directement cette boucle attendra la prochaine version.